Numerical Methods for the Vlasov-Poisson System: 1D Multistream vs 2D Phase Space Discretization

Sania Singh

September 12, 2025

Implemented with Firedrake

Outline

Introduction to Vlasov Equation

From Boltzmann to Collisionless Kinetic Theory

The **Boltzmann equation** describes the evolution of the distribution function $f(\mathbf{x}, \mathbf{v}, t)$:

$$\frac{\partial f}{\partial t} + \mathbf{v} \cdot \nabla_{\mathbf{x}} f + \mathbf{a} \cdot \nabla_{\mathbf{v}} f = C[f] \tag{1}$$

Term-by-Term Explanation:

- $f(\mathbf{x}, \mathbf{v}, t)$: particle distribution function in phase space
- $\mathbf{v} \cdot \nabla_{\mathbf{x}} f$: free streaming in space
- $\mathbf{a} \cdot \nabla_{\mathbf{v}} f$: acceleration in velocity
- $C[f]$: collision operator

In the **collisionless limit** where $C[f] \equiv 0$, we obtain the **Vlasov equation**:

$$\frac{\partial f}{\partial t} + \mathbf{v} \cdot \nabla_{\mathbf{x}} f + \mathbf{a} \cdot \nabla_{\mathbf{v}} f = 0 \tag{2}$$

Particle Method Ansatz: 1D (Part 1)

Velocity Discretization Strategy

Key Idea: Express the distribution function as a sum of delta functions in velocity:

$$f(x, v, t) = \sum_{i=1}^{N} f_i(x, t)\, \delta(v - v_i(x, t)) \tag{3}$$

where:

- $f_i(x, t)$: particle weights (vary in space and time)
- $v_i(x, t)$: particle velocities (functions of space and time)
- N: number of "streams"

This approach discretizes velocity space using N streams while keeping spatial coordinates continuous. Each stream carries weight f_i and moves with velocity v_i.

Particle Method Ansatz: 1D (Part 2)

Evolution Equations & Poisson Coupling

Evolution Equations:

$$\frac{\partial v_i}{\partial t} = -\frac{q}{m}\frac{\partial \phi}{\partial x} \quad \text{(velocity evolution)} \tag{4}$$

$$\frac{\partial f_i}{\partial t} + \frac{\partial}{\partial x}(v_i f_i) = 0 \quad \text{(continuity for weights)} \tag{5}$$

Poisson Coupling: The electric potential satisfies:

$$\frac{\partial^2 \phi}{\partial x^2} = -\frac{q}{\varepsilon_0}\rho(x,t) \tag{6}$$

where the charge density is:

$$\rho(x,t) = \sum_{i=1}^{N} f_i(x,t) \tag{7}$$

This creates a **self-consistent** system: particles generate field, field affects motion.

Multistream Method: The "Streams" Concept

Discretizing Velocity Space with Computational Particles

What is a "Stream"?

- A computational particle carrying weight $f_i(x,t)$ and velocity $v_i(x,t)$
- Each stream represents a slice of the velocity distribution
- M streams $\Rightarrow$ M-point approximation in velocity space

From Vlasov to Streams:

Original Vlasov equation (continuous in v):

$$\frac{\partial f}{\partial t} + v\frac{\partial f}{\partial x} - \frac{\partial \phi}{\partial x}\frac{\partial f}{\partial v} = 0 \tag{8}$$

Multistream approximation:

$$f(x,v,t) = \sum_{i=1}^{M} f_i(x,t)\,\delta(v - v_i(x,t)) \tag{9}$$

Key Insight: Transform PDE into M coupled 1D problems!

Discontinuous Galerkin for Stream Transport

Why DG Works Well for Advection

Each Stream Satisfies:

$$\frac{\partial q_i}{\partial t} + \frac{\partial}{\partial x}(v_i q_i) = 0 \tag{10}$$

What is DG?

- Functions can be **discontinuous** between elements
- No continuity constraints at element boundaries
- Each element is independent

Why DG for Transport?

- Transport naturally creates sharp gradients
- DG handles shocks and discontinuities well
- Local conservation properties
- Stable for hyperbolic problems

Firedrake Implementation: How We Do It

Building Solvers for the Multistream Method

Function Spaces in Firedrake:

- Charge densities q_i: DG1 space (allows discontinuities)
- Velocities v_i: CG1 space (continuous functions)
- Electric potential ϕ: CG1 space (continuous)

Solver Construction:

- **Advection solvers:** One for each stream's q_i
- **Poisson solver:** Computes ϕ from total charge
- **Velocity solvers:** Update each stream's v_i

Time Stepping: Why SSPRK3?

Solving the Coupled q and v Evolution

Our System to Solve:

- Charge densities $q_i(x, t)$ evolve via advection
- Velocities $v_i(x, t)$ evolve via acceleration
- Electric field $\phi(x, t)$ couples everything together

Why SSPRK3 Works Well:

- **Strong Stability Preserving:** Handles hyperbolic transport
- **Multi-stage:** Maintains coupling between q_i and v_i
- **Robust:** Prevents oscillations in sharp gradients
- **Efficient:** Third-order accuracy with stability

The Challenge:

- M streams all coupled through electric field
- Advection equations are hyperbolic
- Need stable, accurate time integration

SSPRK3 Time Stepping Algorithm

Three-Stage Runge-Kutta Method

Strong Stability Preserving RK3: Three-stage time stepping algorithm

Stage 1:

$$q_i^1 = q_i^n + dt \cdot F_q(q_i^n, v_i^n) \tag{11}$$

$$v_i^1 = v_i^n + dt \cdot F_v(v_i^n, \phi^n) \tag{12}$$

Stage 2:

$$q_i^2 = \frac{3}{4} q_i^n + \frac{1}{4}(q_i^1 + dt \cdot F_q(q_i^1, v_i^1)) \tag{13}$$

$$v_i^2 = \frac{3}{4} v_i^n + \frac{1}{4}(v_i^1 + dt \cdot F_v(v_i^1, \phi^1)) \tag{14}$$

Stage 3:

$$q_i^{n+1} = \frac{1}{3} q_i^n + \frac{2}{3}(q_i^2 + dt \cdot F_q(q_i^2, v_i^2)) \tag{15}$$

$$v_i^{n+1} = \frac{1}{3} v_i^n + \frac{2}{3}(v_i^2 + dt \cdot F_v(v_i^2, \phi^2)) \tag{16}$$

Initialization & Evolution

From Setup to Plasma Dynamics

Stream Initialization:

- Use **Hermite quadrature** to discretize velocity space
- M quadrature points become stream velocities v_i
- Quadrature weights scaled by spatial distribution

Initial Conditions:

- **Background:** Maxwellian distribution $\frac{e^{-v^2/2}}{\sqrt{2\pi}}$
- **Perturbation:** Small spatial variation $(1 + A\cos(kx))$
- **Parameters:** $A = 0.05$, $k = 0.5$ (weak instability)

After Time Evolution:

- **Updated charges:** $q_i(x, t)$ for each stream
- **Updated velocities:** $v_i(x, t)$ for each stream

Result: Evolved `q_list` and `u_list` representing plasma state

2D Method: Direct Phase Space Discretization

Extruded Mesh for x-v Domain

Alternative Approach: Direct discretization of (x, v) phase space

Extruded Mesh Construction:

- **Base mesh:** 1D periodic mesh in $x \in [0, 8\pi]$
- **Extrusion:** Add layers in velocity direction $v \in [-5, 5]$
- **Result:** 2D mesh covering full phase space

Direct Discretization:

- Distribution function $f(x, v, t)$ on 2D mesh
- No velocity approximation - full kinetic resolution
- DG elements handle transport in phase space

Comparison:

- **Multistream:** M streams, reduced problem size
- **2D method:** Full resolution, higher computational cost

Moments: Comparing the Methods

Velocity Integrals of the Distribution

What are Moments? Moments are velocity integrals of the distribution function:

$$M(x,t) = \int w(v) f(x,v,t)\, dv \tag{17}$$

Weight Functions:

- $w(v) = 1$: **Density** (total charge at each point)
- $w(v) = v$: **Momentum density** (average velocity)
- $w(v) = v^2$: **Energy density** (kinetic energy)

Why Moments for Comparison?

- Reduce 2D distribution to 1D functions
- Physically meaningful quantities
- Allow direct comparison between methods

Mesh Immersion & Interpolation

Bridging Different Discretizations

The Challenge:

- Multistream: moments defined on 1D spatial mesh
- 2D method: moments defined on 2D phase space mesh
- Need common mesh for comparison

Creating an immersed mesh:

- Start with 1D spatial mesh from multistream method
- Create "line mesh": $(x, v = 0)$ embedded in 2D space
- Represents spatial variation with fixed velocity coordinate

Interpolation Process:

- Compute moments from 2D distribution function
- Transfer 2D moment data to immersed line mesh
- Interpolate multistream moments to same mesh structure
- Both methods now have moments on identical mesh

Firedrake Tools: Built-in interpolation and mesh manipulation capabilities

Mesh Immersion & Interpolation

Basic Concepts

Mesh Immersion: Taking a 1D mesh and embedding it in 2D space. For example, a line mesh (x) becomes $(x, v = 0)$ in the (x, v) plane.

Interpolation: Transferring function values from one mesh to another. Firedrake automatically handles the geometric transformations needed.

Why We Need This: Our multistream method works on 1D spatial mesh, while the 2D method uses full phase space mesh. To compare moments, we need a common mesh structure.

The Process: We create an immersed line mesh from the 1D spatial mesh, then interpolate both methods' moments onto this common mesh for comparison.

Checkpointing & Comparison Strategy

Systematic Method Comparison

Checkpointing Strategy:

- **Initial checkpoint:** Save state before time evolution
- **Final checkpoint:** Save state after time evolution
- Both multistream and 2D methods use same checkpointing

Moment Computation:

- **2D method:** Compute $\int v^2 f(x, v, t)\, dv$ directly
- **Multistream:** Compute $\sum_i v_i^2 q_i(x, t)$ from streams
- Both give spatial functions of kinetic energy density

Comparison Process:

- Transfer both moments to common immersed mesh
- Compute relative L2 errors between methods
- Test multiple values of M (number of streams)
- Analyze both initial and final time checkpoints

Goal: Compare methods through moments and analyze convergence rate

Error Analysis & Convergence

Quantifying Method Accuracy

Error Computation: Relative L2 error between moment functions:

$$\text{Error} = \frac{\|M_{\text{multistream}} - M_{\text{2D}}\|}{\|M_{\text{2D}}\|} \tag{18}$$

where M represents the v^2-weighted moments.

Expected Behavior: As number of streams M increases, multistream approximation should converge to 2D method. Errors should decrease systematically.

Test Range: M values from 2 to 51 streams, testing both initial and final time checkpoints.

Results: